SC3020-CE4301-CZ4031: Project 1

Prepared by Yufei Tao
Feb 1, 2026

DEADLINE: 11:59pm, Mar 14, 2026.

1 Main Objective

This project is designed to deepen your understanding of storage, buffering, and indexing through authentic experience on a real database system.

2 Setup

This guide assumes that you are using Windows, which is the operating system installed on all the machines in the CCDS labs.

First, download PostgreSQL at `https://www.postgresql.org/download` and install it. Make sure `pgAdmin4` is working — this is a utility program that allows you to access and manage the database with a friendly GUI.

Second, download the file `proj1-data.zip` from NTULearn (look for `Project 1` and then `Data for Project 1`). Unzip the file and you should see 8 `csv` files. These files were generated according to the TPC-H benchmark (see `https://www.tpc.org/tpch` for the details of TPC-H).

Third, download the file `table-creation.txt` from NTULearn (look for `Project 1` and then `Table Creation Statements`). Use `pgAdmin4` to create a new database in PostgreSQL, and then use the statements in `table-creation.txt` to create 9 relations inside that database. Make sure that you see the following relations in `pgAdmin4`:

- `customer`
- `lineitem`
- `nation`
- `orders`
- `part`
- `partsupp`
- `region`
- `supplier`

Fourth, use `pgAdmin4` to import data into the first 8 relations from the `csv` files you obtained earlier. The file names match the relation names in a straightforward manner (e.g., file `customer.csv` is for relation `customer`). Note the foreign key constraints on the relations; choose an appropriate import order that respects these constraints.

3 Disk Storage

3.1 Task 1

Use the following command to find the block size in your system:

```
show block_size;
```

3.2 Task 2

Use the following command to find the length (in the number of bytes) of the file storing the relation `customer`:

```
select pg_relation_size('customer');
```

Calculate the number of blocks the file occupies. In the same fashion, obtain the number of blocks occupied by every other relation.

3.3 Task 3

The `d+` command allows you to find the basic information of each relation. From online sources, figure out the syntax of `d+` and answer the following questions regarding the output of `d+`:

- What are the meanings of `plain`, `main`, and `extended` under the `Storage` column?
- What is the meaning of `Access method: heap`?

4 Column 'ctid'

Every relation has a hidden column named `ctid` that is added by PostgreSQL automatically when the relation is created.

4.1 Task 4

Execute the command:

```
select ctid, * from customer;
```

Figure out from online sources the meaning of a ctid value — which is a pair — from online sources. Now, execute the following command:

```
select ctid, * from lineitem;
```

Explain why both queries return a tuple with ctid $(0, 1)$.

4.2 Task 5

For each of the 8 relations, explain how the tuples are ordered in the file storing the relation. Justify your answers.

4.3 Task 6

Execute the following commands:

```
create index idx_customer_phone on customer(c_phone);
cluster customer using idx_customer_phone;
```

How are the `customer` tuples ordered in the file now? Justify your answer.

Before proceeding, execute:

```
drop index idx_customer_phone;
```

5 Buffering

Familiarize yourself with the use of `explain` and `analyze` in PostgreSQL:

- Putting `explain` before an SQL query prompts the system to explain the execution plan selected for the query. For example, try:

  ```
  explain select * from lineitem;
  ```

  to see what happens.

- Putting `explain analyze` before an SQL query prompts the system to (i) explain the execution plan selected for the query and (ii) physically execute the plan to obtain the actual performance statistics. For example, try:

  ```
  explain analyze select * from lineitem;
  ```

  to see what happens.

5.1 Task 7

Use the following command to find the buffer pool size in your system:

```
show shared_buffers;
```

The size is shown in the number of bytes. Convert it to the number of frames (using the block size found in Task 1).

5.2 Task 8

Perform a fresh boot of your system. Execute the command:

```
explain analyze select * from lineitem;
```

How many buffer hits and misses were there? Note: you should look at the numbers reported for `Seq Scan`, rather than for `Planning` (the numbers reported for `Planning` indicate the cost of query optimization, not the cost of query execution).

Now run the query for the second time:

```
explain analyze select * from lineitem;
```

How many buffer hits and misses were there? Provide an explanation for those numbers — for this purpose, consult online sources to understand how PostgreSQL manages the buffer pool in performing a relation scan.

5.3 Task 9

Before doing this task, make sure that there is a B-tree on the primary key of `lineitem` relation (this should have been automatically created by the system) — use the `d+` command for this purpose.

Execute the command:

```
explain analyze select * from lineitem where l_orderkey = 70;
```

How many buffer hits and misses were there? Now execute the same command again:

```
explain analyze select * from lineitem where l_orderkey = 70;
```

How many buffer hits and misses were there? Provide an explanation.

Now execute the same command again:

```
explain analyze select * from lineitem where l_orderkey = 71;
```

How many buffer hits and misses were there? Provide an explanation.

6 Indexing

The tasks in this section are designed to familiarize you with the B-tree and hash indexes in PostgreSQL.

6.1 Task 10

Use `d+` command to find the name of the B-tree index on the primary key of `customer`. Henceforth, we assume that the name is `customer_pkey` (if this is not the index name in your case, modify the statement accordingly). Run

```
cluster customer using customer_pkey;
```

Now, execute the command:

```
explain analyze select * from customer where c_custkey = 100;
```

Did the execution use the B-tree? How many buffer hits and misses were there?

Run the following to create a hash index:

```
create index idx_customer_custkey_hash on customer using hash (c_custkey);
```

Execute again:

```
explain analyze select * from customer where c_custkey = 100;
```

Did the execution use the hash index? Provide an explanation.

Let us continue with this query:

```
explain analyze select * from customer where c_custkey >= 200 and c_custkey <= 201;
```

Did the execution use the hash index? Provide an explanation.

Before proceeding, run:

```
drop index idx_customer_custkey_hash;
```

6.2 Task 11

In this task, you will learn about PostgreSQL's "bitmap index scan" and "bitmap heap scan". Before proceeding, please note that all B-tree indexes in PostgreSQL are dense indexes — each "data element" (i.e., an entry in a leaf node) is associated with the `ctid` value (i.e., a pointer) of the corresponding tuple.

Execute the following commands:

```
cluster customer using customer_pkey;
create index idx_customer_phone on customer(c_phone);
```

The first command assumes `customer_pkey` to be the default index created on `customer`. If not, replace it with the corresponding index name.

Run

```
explain analyze select * from customer;
```

How did the database execute the query? How many buffer hits and misses?

Run

```
explain analyze select * from customer where c_phone >= '2';
```

How did the database execute the query? How many buffer hits and misses?

Run

```
explain analyze select * from customer where c_phone >= '2' and c_phone <= '21';
```

Explain:

- how `Bitmap Index Scan` is performed here (consulting online resources if needed);
- how `Bitmap Heap Scan` is performed here (consulting online resources if needed);
- the numbers of buffer hits and misses for each of the above two "scans";
- why the database decided to use `idx_customer_phone` for this query, and why it chose not to do so for the previous query (with the condition `c_phone >= '2'`).

Run

```
cluster customer using idx_customer_phone;
```

Then re-run the query

```
explain analyze select * from customer where c_phone >= '2' and c_phone <= '21';
```

How many buffer hits and misses? How do they compare with the numbers you obtained earlier? Provide an explanation.

Before proceeding, execute

```
drop index idx_customer_phone;
```

7 (Slightly) Sophisticated Query Processing

The tasks in this section enable you to look deeper into the factors affecting how a database decides to process a query.

7.1 Task 12

Execute the following commands:

```
create index idx_customer_phone on customer(c_phone);
cluster customer using idx_customer_phone;
```

Run

```
explain analyze select * from customer where c_phone >= '2' and c_phone <= '3' and
c_custkey >= 10000 and c_custkey <= 11000;
```

How was the query executed? Provide an explanation.

Now, run

```
 explain analyze select * from customer where c_phone >= '2' and c_phone <= '20-170'
and c_custkey >= 10000 and c_custkey <= 11000;
```

How was the query executed? Provide an explanation.

Before proceeding, execute

```
drop index idx_customer_phone;
```

7.2 Task 13

Before proceeding, make sure that only the primary-key index exists on the `lineitem` table.

Run

```
explain analyze select * from lineitem where l_orderkey = 2000;
```

How many levels does the B-tree (i.e., the primary-key index) have? Justify your answer.

Run

```
explain analyze select * from lineitem where l_orderkey <= 2000 and l_partkey >= 10000
and l_partkey <= 10010;
```

Describe how the query was executed based on your best understanding.

Run

```
explain analyze select * from lineitem where l_partkey >= 10000 and l_partkey <= 10010;
```

Describe how the query was executed based on your best understanding.

Run

```
explain analyze select l_partkey from lineitem where l_partkey >= 10000 and l_partkey
<= 10010;
```

Describe how the query was executed based on your best understanding.

Run

```
explain analyze select * from lineitem where l_partkey >= 10000 and l_partkey <= 10000;
```

Describe how the query was executed based on your best understanding.

8 Index Maintenance

The tasks below allow you to experience the overhead of index maintenance. You will need to write Python code to send database statements to PostgreSQL. Go to NTULearn and download the file `ex.py` (look for `Project 1` and then `Python example`), which contains a small example to get you started.

8.1 Task 14

Create a table:

```
create table t1(a integer, b integer, c integer, d integer, e integer,
primary key(a, b, c, d, e));
```

If the table already exists, delete it first using the `drop` command. Now, create 5 indexes:

```
create index idx_t1_a on t1(a);
create index idx_t1_b on t1(b);
create index idx_t1_c on t1(c);
create index idx_t1_d on t1(d);
create index idx_t1_e on t1(e);
```

Write a Python program to execute the following statement

```
insert into t1 select (floor(random()*10000)+1)::int, (floor(random()*10000)+1)::int,
(floor(random()*10000)+1)::int, (floor(random()*10000)+1)::int,
(floor(random()*10000)+1)::int from generate_series(1, 1000000);
```

Measure the running time of the above statement inside the Python program and display it on screen. Before closing your connection to PostgreSQL, remember to issue a `commit` — otherwise, the 1,000,000 tuples inserted by the above statement may "disappear" from the table.

To learn about the overhead of remote connections, write a program to insert another 1,000,000 tuples into the `t1` relation in a for-loop that looks like:

```
for i in range(1000000):
     a = random.randint(1, 10000)
     b = random.randint(1, 10000)
     c = random.randint(1, 10000)
     d = random.randint(1, 10000)
     e = random.randint(1, 10000)
#    issue a statement here to insert a tuple (a, b, c, d, e)
```

Measure the total time of the 1,000,000 insertions inside the Python program and display it on screen.

8.2 Task 15

Create a table:

```
create table t2(a integer, b integer, c integer, d integer, e integer, primary key(a,
b, c, d, e));
```

If the table already exists, delete it first using the `drop` command. Make sure that no extra indexes are created on `t2`.

Write a Python program to execute the following statement

```
insert into t1 select (floor(random()*10000)+1)::int, (floor(random()*10000)+1)::int,
(floor(random()*10000)+1)::int, (floor(random()*10000)+1)::int,
(floor(random()*10000)+1)::int from generate_series(1, 1000000);
```

Measure the running time of the above statement inside the Python program and display it on screen. Before closing your connection to PostgreSQL, remember to issue a `commit` — otherwise, the 1,000,000 tuples inserted by the above statement may "disappear" from the table.

How does the running time compare to the one you obtained in Task 14? Provide an explanation.

9 What to Submit

Each project group should submit a report in PDF format. The report must contain a separate section for every task. Specifically, each section should include

- the necessary screenshots to demonstrate how the corresponding task was carried out;
- convincing explanations for the answers to the questions posed in this document. Relate your explanations to our lecture notes and base your explanations on evidence.

For Tasks 14 and 15, the report must include the complete python code written.

In addition, the report must include a declaration about each group member's contributions, including a contribution percentage for each member (e.g., if your group has 5 members and everyone has contributed equally, you should put down 20% for each person).

Even though this is a group project, every student is expected to understand the contents of the *entire* report. For the best training, each student should carry out all the tasks individually at least once — possibly by learning from other group members.

One submission per group is sufficient. The submission deadline is **11:59pm, Mar 14, 2026**.